TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI TP.HCM

BÁO CÁO CUỐI KÌ

ĐỀ TÀI: BỘ GIẢI BÀI TOÁN 8-PUZZLE / 15-PUZZLE

MÃ HỌC PHẦN: 012012103307 (NHÓM 5)

Giảng viên hướng dẫn: Bùi Trọng Hiếu

Sinh viên thực hiện: Trần Hồng Sơn

Trần Duy Tân

Hoàng Công Minh Tâm

Võ Thị Anh Tú

Nguyễn Hồ Diễm My

Lê Quốc Thắng

TP.HCM – 11/ 2026

LỜI CẢM ƠN

Trong quá trình thực hiện đồ án môn Trí tuệ nhân tạo với đề tài *"PuzzleSolver-AI – Bộ giải bài toán 8-Puzzle / 15-Puzzle bằng thuật toán A*", nhóm đã có cơ hội vận dụng những kiến thức lý thuyết đã được học vào việc nghiên cứu, phân tích và xây dựng một chương trình giải quyết bài toán tìm kiếm trạng thái.

Nhóm xin gửi lời cảm ơn chân thành đến giảng viên hướng dẫn đã truyền đạt kiến thức, định hướng và hỗ trợ nhóm trong quá trình học tập cũng như thực hiện đồ án.

Thông qua đề tài, nhóm đã tìm hiểu và vận dụng các kiến thức về không gian trạng thái, thuật toán tìm kiếm A*, hàm Heuristic, cấu trúc dữ liệu, phương pháp kiểm thử và đánh giá hiệu năng thuật toán. Bên cạnh đó, việc xây dựng giao diện trực quan bằng thư viện Pygame giúp nhóm hiểu rõ hơn về cách kết hợp giữa thuật toán xử lý và khả năng trình diễn kết quả.

Mặc dù đã cố gắng hoàn thiện sản phẩm và báo cáo, do thời gian thực hiện cũng như kinh nghiệm còn hạn chế, đồ án khó tránh khỏi những thiếu sót. Nhóm rất mong nhận được những ý kiến đóng góp từ giảng viên để có thể hoàn thiện sản phẩm và nâng cao kiến thức chuyên môn.

Một lần nữa, nhóm xin chân thành cảm ơn!

BẢNG PHÂN CÔNG THÀNH VIÊN TRONG NHÓM

STT	MSSV	HỌ VÀ TÊN	PHÂN CÔNG	% HOÀN THÀNH
1	**060206012202**	Trần Hồng Sơn	Điều khiển giao diện Pygame, kiến trúc State Machine, bảng HUD và tích hợp hệ thống.	100%
2	**083206003584**	Trần Duy Tân	Cài đặt 2 hàm Heuristic (Manhattan, Misplaced) và thuật toán sinh test case Random Walk có kiểm soát độ khó.	100%
3	**045206008142**	Hoàng Công Minh Tâm	Biểu diễn trạng thái Tuple, sinh trạng thái con láng giềng, đếm Inversion và kiểm tra khả giải.	100%
4	**052306001040**	Võ Thị Anh Tú	Xây dựng hệ thống đo lường hiệu năng tự động (Benchmark), xuất CSV, phân tích Pandas và vẽ biểu đồ Matplotlib.	100%
5	**075306001575**	Nguyễn Hồ Diễm My	Thiết kế hiển thị bàn cờ responsive bo góc, hiệu ứng 3D đổ bóng và hoạt ảnh trượt mượt mà Ease-Out Cubic.	100%
6	**079206003650**	Lê Quốc Thắng	Cài đặt thuật toán A* tổng quát, hàng đợi Min-Heap (heapq), cơ chế an toàn Timeout & Node Limit.	100%

MỤC LỤC

DANH MỤC CÁC BẢNG i
DANH MỤC CÁC HÌNH, SƠ ĐỒ ii
LỜI MỞ ĐẦU iii

CHƯƠNG 1: Giới thiệu tổng quan 1
1.1. Giới thiệu đề tài 1
1.2. Lý do chọn đề tài 1
1.3. Mục tiêu thực hiện 1
1.4. Phạm vi thực hiện 1

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT 2
2.1. Không gian trạng thái và Biểu diễn bài toán N-Puzzle 2
2.1.1. Mô hình hóa bài toán 2
2.3. Lý thuyết về Điều kiện Khả giải (Solvability) của N-Puzzle 2
2.3.1. Khái niệm Nghịch thế (Inversions) 3
2.3.2. Định lý khả giải 3
2.4. Thuật toán tìm kiếm A* (A-Star Search) 3
2.4.1. Nguyên lý hoạt động 3
2.4.2. Cấu trúc dữ liệu cài đặt 3
2.4.3. Tính đúng đắn và tối ưu (Optimality) 4
2.4.4. Cơ chế an toàn (Safety Limits) trong bài toán lớn (15 – Puzzle) 4
2.5. Hàm Heuristic 4
2.5.1. Hai hàm Heuristic đã cài đặt và chứng minh tính chất toán học 4
2.5.1.1. Hàm Heuristic 1: Misplaced Tiles (h1 - Đếm số ô đặt sai vị trí) 5

2.5.1.2. Hàm Heuristic 2: Manhattan Distance (h2 – Tổng khoảng cách theo hàng và cột) 5
2.5.1.3. Ví dụ minh họa h1(n) và h2(n) trên bàn cờ 8 – Puzzle 5
2.5.1.4. Chứng minh tính Chấp nhận được (Admissible) và Quan hệ Trội (Dominance) 6

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG 6
3.1. Tổng quan dự án PuzzleSolver – AI 6
3.2. Phân tích yêu cầu hệ thống 7
3.2.1. Yêu cầu chức năng 7
3.2.2. Yêu cầu phi chức năng 8
3.3. Phân tích các thành phần chức năng 8
3.3.1. Thành phần xử lý cốt lõi 8
3.3.2. Thành phần Heusistic 8
3.3.3. Thành phần sinh dữ liệu 8
3.3.4. Thành phần đánh giá hiệu năng 8
3.3.5. Thành phần giao diện 9
3.4. Thiết kế kiến trúc hệ thống 9
3.4.1. Sơ đồ kiến trúc tổng thể 9
3.4.2. Sơ đồ luồng xử lý 10
3.5. Thiết kế module chương trình 10
3.6. Thiết kế dữ liệu đầu vào 11
3.7. Thiết kế dữ liệu đầu ra 11
3.8. Tổng kết chương 11

CHƯƠNG 4: XÂY DỰNG VÀ TRIỂN KHAI CHƯƠNG TRÌNH 11
4.1. Môi trường và công cụ phát triển 11
4.2. Xây dựng thuật toán A* 11
4.3. Xây dựng hàm Misplaced Tiles 12
4.4. Xây dựng hàm Misplaced Tiles 12
4.5. Chứng minh tính chất của hai hàm Heuristic 13
4.5.1. Misplaced Tiles là hàm chấp nhận được 13
4.5.2. Manhattan Distance là hàm chấp nhận được 13
4.5.3. Quan hệ trội 13
4.6. Xây dựng bộ sinh dữ liệu Random Walk 13
4.6.1. Lý do sử dụng Random Walk 13

4.6.2. Nguyên lý hoạt động 14
4.6.3. Tính tái lập 14
4.7. Xây dựng chức năng Benchmark 14
4.8. Xây dựng giao diện trực quan bằng Pygame 15
4.8.1. Hiển thị trạng thái bàn cờ 15
4.8.2. Thiết kế màu sắc 15
4.9. Xây dựng hoạt ảnh trượt ô 16
4.10. Tích hợp các thành phần hệ thống 16
4.11. Các thao tác điều khiển 17

CHƯƠNG 5: KIỂM THỬ, THỰC NGHIỆM VÀ ĐÁNH GIÁ 17
5.1. Mục tiêu thực nghiệm 17
5.2. Phương pháp kiểm thử 18
5.3. Các tiêu chí đánh giá 18
5.3.1. Số node mở rộng 18
5.3.2. Độ dài lời giải 18
5.3.3. Thời gian thực thi 18
5.4. Kết quả thực nghiệm đối với 8 – Puzzle 19
5.4.1. Phân tích số node 19
5.4.2. Phân tích thời gian thực thi 19
5.5. Kết quả thực nghiệm đối với 15 – Puzzle 20
5.5.1. Phân tích số node 20
5.5.2. Phân tích thời gian thực thi 20
5.6. So sánh tổng hợp hai hàm Heuristic 21
5.6.1. So sánh số node mở rộng 21
5.6.2. So sánh thời gian thực thi 21
5.7. Đánh giá độ dài thời gian 22
5.8. Đánh giá giao diện trực quan 22
5.9. Nhận xét chung 23

CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 23
6.1. Kết quả đạt được 23
6.2. Ưu điểm của hệ thống 24
6.3. Hạn chế của hệ thống 24
6.4. Hướng phát triển 25

6.5. Kết luận chung 25

DANH MỤC CÁC BẢNG

Bảng 1: Yêu cầu chức năng 7
Bảng 2: Các chức năng xử lý trạng thái và A* được cung cấp bởi phần xử lý cốt lõi của dự án 10
Bảng 3: Các công cụ và thư viện 11
Bảng 4: Màu sắc của các ô 15
Bảng 5: Các thao tác để điều khiển 17
Bảng 6: Kết quả trung bình của 8-Puzzle. 19
Bảng 7: Kết quả trung bình của 15-Puzzle. 20
Bảng 8: So sánh số node mở rộng 21
Bảng 9: So sánh thời gian thực thi 22

DANH MỤC CÁC HÌNH, SƠ ĐỒ

Hình 1: Sơ đồ thể hiện mối liên hệ giữa phần xử lý trạng thái, tìm kiếm, Heuristic và giao diện trực quan.....9
Hình 2: Sơ đồ luồng xử lý 10

LỜI MỞ ĐẦU

Trong những năm gần đây, Trí tuệ nhân tạo (Artificial Intelligence – AI) ngày càng phát triển và được ứng dụng rộng rãi trong nhiều lĩnh vực đời sống. Một trong những nền tảng quan trọng của trí tuệ nhân tạo là về các thuật toán tìm kiếm giúp máy tính có khả năng phân tích các không gian trạng thái và lựa chọn các hướng giải quyết tìm ra lời giải phù hợp chi những bài toán cụ thể. Trong đó, thuật toán tìm kiếm A* (A – Star) là một thuật toán tiêu biểu.

Báo cáo này là kết quả của quá trình tìm hiểu, nghiên cứu và thực hiện đề tài. Dù nhóm đã cố gắng hoàn thành các nội dung một cách tốt nhất, nhưng do kiến thức và kinh nghiệm thực tế còn hạn chế, báo cáo chắc chắn không tránh khỏi những thiếu sót. Nhóm rất mong nhận được sự góp ý và hướng dẫn từ giảng viên để có thể bổ sung, hoàn thiện kiến thức và cải thiện kết quả thực hiện.

Kết cấu bài báo cáo gồm bốn chương:

- Chương 1: Giới thiệu tổng quan
- Chương 2: Cơ sở lý thuyết về thuật toán
- Chương 3: Thiết kế hệ thống
- Chương 4: Xây dựng và triển khai chương trình
- Chương 5: Kiểm thử, thực nghiệm và đánh giá
- Chương 6: Kết luận và hướng phát triển

Thông qua việc nghiên cứu và xây dựng hệ thống PuzzleSolver-AI, nhóm không chỉ có cơ hội vận dụng các kiến thức lý thuyết về thuật toán tìm kiếm A* vào giải quyết một bài toán thực tế, mà còn hiểu rõ hơn về cách biểu diễn không gian trạng thái, xây dựng hàm Heuristic và tối ưu quá trình tìm kiếm lời giải. Qua đó, đề tài góp phần củng cố kiến thức nền tảng về Trí tuệ nhân tạo và mở ra hướng tiếp cận thực tiễn trong việc ứng dụng các thuật toán tìm kiếm để giải quyết những bài toán phức tạp hơn.

CHƯƠNG 1: Giới thiệu tổng quan

1.1. Giới thiệu đề tài

Trong lĩnh vực Trí tuệ nhân tạo, các thuật toán tìm kiếm đóng vai trò quan trọng trong việc giúp máy tính tìm ra lời giải cho những bài toán cụ thể. Trong đó, thuật toán A* là một phương pháp tìm kiếm có thông tin, kết hợp chi phí thực tế và chi phí ước lượng để lựa chọn hướng tìm kiếm phù hợp.

Từ những kiến thức đã học ở môn Trí tuệ nhân tạo, nhóm quyết định lựa chọn đề tài ***"PuzzleSolver-AI – Bộ giải bài toán 8-Puzzle / 15-Puzzle bằng thuật toán A*".*** Đề tài tập trung xây dựng chương trình bằng Python nhằm tự động giải hai dạng trò chơi xếp hình 8 – Puzzle và 15 – Puzzle. Chương trình sử dụng thuật toán A* kết hợp với hai phương pháp ước lượng Misplaced Tiles và Manhattan Distance để tìm đường đi từ trạng thái ban đầu đến trạng thái đích.

1.2. Lý do chọn đề tài

Nhóm lựa chọn đề tài này vì mong muốn vận dụng được những kiến thức đã học vào một bài toán thực tế, qua đó có thể hiểu rõ hơn về cách hoạt động của thuật toán tìm kiếm A*. Mặc dù 8 – Puzzle và 15 – Puzzle có luật chơi đơn giản, số lượng trạng thái có thể xuất hiện rất lớn đặc biệt đối với 15 – Puzzle. Bên cạnh đó, việc áp dụng hai phương pháp Heuristic giúp nhóm có cơ hội so sánh hiệu quả tìm kiếm và rèn luyện tư duy thuật toán.

1.3. Mục tiêu thực hiện

- Tìm hiểu nguyên lý hoạt động của thuật toán A* và các phương pháp Heuristic
- Xây dựng chương trình có khả năng giải bài toán 8 – Puzzle và 15 – Puzzle
- Áp dụng hai phương pháp ước lượng Misplaced Tiles và Manhattan Distance
- Kiểm tra tính khả giải của trạng thái đầu vào và tìm đến trạng thái đích

1.4. Phạm vi thực hiện

Đề tài giới hạn trong việc giải hai bài toán 8 – Puzzle (bảng 3x3) và 15 – Puzzle (bảng 4x4), sử dụng thuật toán A* làm phương pháp tìm kiếm chính. Chương trình được xây dựng bằng ngôn ngữ Python, áp dụng hai hàm Heuristic là Misplaced Tiles và Manhattan Distance. Đề tài tập trung vào mục đích học tập, thực hành và đánh giá thuật toán A*, chưa mở rộng sang các thuật toán khác.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Không gian trạng thái và Biểu diễn bài toán N-Puzzle

2.1.1. Mô hình hóa bài toán

Bài toán 8-puzzle / 15-puzzle: Là bài toán trò chơi trượt ô kinh điển trong Trí tuệ nhân tạo. Bàn cờ gồm một lưới kích thước N × N (với N = 3 cho 8-puzzle và N = 4 cho 15-puzzle), chứa N^2 - 1 ô số từ 1 đến N^2 - 1 và 1 ô trống (kí hiệu là 0).
Trạng thái đích (Goal State): Trạng thái mà các ô số được sắp xếp theo thứ tự tăng dần từ trái sang phải, từ trên xuống dưới, và ô trống nằm ở góc dưới cùng bên phải:

- Với 3 × 3: (1, 2, 3, 4, 5, 6, 7, 8, 0)
- Với 4 × 4: (1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 0).

Kích thước không gian trạng thái:

- Đối với 8-puzzle: Tổng số hoán vị là 9! = 362,880. Do tính chất khả giải, chỉ có một nửa trạng thái là đi đến được đích: 9! / 2 = 181,440 trạng thái khả giải.
- Đối với 15-puzzle: Tổng số hoán vị là 16! ≈ 2.09×10^{13}. Số trạng thái khả giải: 16! / 2 ≈ 1.046×10^{13} trạng thái.

Sự bùng nổ tổ hợp này đòi hỏi thuật toán tìm kiếm phải có hàm Heuristic định hướng tốt để không bị cạn kiệt bộ nhớ.

2.2. Biểu diễn trạng thái trong chương trình

Trạng thái được biểu diễn dưới dạng một Tuple gồm N^2 phần tử số nguyên bất biến (immutable) trong Python để:

- Tối ưu bộ nhớ so với mảng 2 chiều.
- Cho phép làm Key trong bảng băm (dict và set), giúp việc tra cứu trạng thái đã duyệt đạt độ phức tạp O(1).

Hàm sinh trạng thái kề (get_neighbors): Xác định tọa độ ô trống (r, c) = (blank // N, blank % N) và thực hiện hoán đổi vị trí với các ô kề hợp lệ theo 4 hướng: Lên, Xuống, Trái, Phải (bỏ qua các bước di chuyển vượt biên).

2.3. Lý thuyết về Điều kiện Khả giải (Solvability) của N-Puzzle

Không phải bất kỳ cấu hình ngẫu nhiên nào của bàn cờ cũng có thể giải được về trạng thái đích. Việc kiểm tra tính khả giải trước khi chạy tìm kiếm giúp thuật toán dừng ngay lập tức nếu gặp bài toán vô nghiệm.

2.3.1. Khái niệm Nghịch thế (Inversions)

Xét trạng thái bàn cờ dưới dạng danh sách tuyến tính, bỏ qua ô trống (0). Một cặp ô (a, b) được gọi là một nghịch thế (inversion) nếu: a xuất hiện trước b và a > b

2.3.2. Định lý khả giải

Trường hợp lưới lẻ (N là số lẻ, ví dụ N = 3 trong 8-puzzle):

- Trạng thái giải được khi và chỉ khi tổng số nghịch thế là một số chẵn: Inversions ≡ 0 (mod 2)

Trường hợp lưới chẵn (N là số chẵn, ví dụ N = 4 trong 15-puzzle):

- Tính khả giải phụ thuộc vào cả số nghịch thế và vị trí hàng của ô trống (tính từ dưới lên, bắt đầu từ 1):

Nếu ô trống nằm ở hàng chẵn (tính từ dưới lên): Bài toán giải được khi tổng số nghịch thế là số lẻ.
Nếu ô trống nằm ở hàng lẻ (tính từ dưới lên): Bài toán giải được khi tổng số nghịch thế là số chẵn.

2.4. Thuật toán tìm kiếm A* (A-Star Search)

2.4.1. Nguyên lý hoạt động

A* là thuật toán tìm kiếm đồ thị có thông tin (informed search), kết hợp ưu điểm của thuật toán Dijkstra (tìm đường đi ngắn nhất) và Greedy Best-First Search (định hướng nhanh về đích nhờ heuristic).
Tại mỗi bước, thuật toán chọn trạng thái n có giá trị hàm đánh giá f(n) nhỏ nhất để mở rộng: f(n) = g(n) + h(n).

Trong đó:

- g(n): Chi phí thực tế đã đi từ trạng thái bắt đầu (start state) đến trạng thái hiện tại n (mỗi bước trượt ô có chi phí 1).
- h(n): Chi phí ước lượng từ trạng thái n đến trạng thái đích (Goal State) do hàm Heuristic cung cấp.
- f(n): Ước lượng tổng chi phí của đường đi tối ưu đi qua trạng thái n.

2.4.2. Cấu trúc dữ liệu cài đặt

Để thuật toán chạy với hiệu năng cao nhất:

- Open Set (Priority Queue / Min-Heap): Lưu trữ các node ứng viên chờ mở rộng, được tổ chức dưới dạng cây nhị phân Min-Heap (heapq). Mỗi phần tử là tuple (f_score, counter, state) cho phép lấy ra node có f(n) nhỏ nhất với độ phức tạp O(log K).
- Closed Set (set): Lưu các trạng thái đã được mở rộng nhằm tránh duyệt lại các chu trình lặp.
- Bảng chi phí g (g_score: dict): Lưu chi phí ngắn nhất từng biết để đi tới mỗi trạng thái.
- Bảng truy vết (came_from: dict): Lưu vết node cha của từng trạng thái để phục hồi toàn bộ đường đi nghiệm sau khi chạm tới đích (_reconstruct_path).

2.4.3. Tính đúng đắn và tối ưu (Optimality)

Tính chấp nhận được (Admissibility): Hàm heuristic h(n) được gọi là chấp nhận được nếu nó không bao giờ đánh giá quá cao chi phí thực tế $h^*(n)$ để đến đích: $0 \leq h(n) \leq h^*(n), \ \forall n$.

Khi $h(n)$ thỏa mãn tính admissible (như Manhattan Distance hay Misplaced Tiles), thuật toán A* đảm bảo luôn tìm ra lời giải tối ưu (chuỗi bước đi ngắn nhất).

2.4.4. Cơ chế an toàn (Safety Limits) trong bài toán lớn (15 – Puzzle)

Trong thực tế, khi giải 15-puzzle với các cấu hình bị xáo trộn sâu, không gian tìm kiếm có thể bùng nổ, dẫn đến tràn bộ nhớ RAM hoặc treo máy. Do đó, thuật toán được bổ sung hai tham số an toàn:

- node_limit: Giới hạn tối đa số lượng node được phép mở rộng.
- timeout_seconds: Giới hạn thời gian tối đa cho phép thuật toán chạy.

Nếu chạm một trong hai ngưỡng này, thuật toán sẽ dừng có kiểm soát và trả về lý do cụ thể ("node_limit" hoặc "timeout"), cung cấp số liệu thực nghiệm phục vụ việc đánh giá khả năng mở rộng của mô hình trong báo cáo.

2.5. Hàm Heuristic

2.5.1. Hai hàm Heuristic đã cài đặt và chứng minh tính chất toán học

Trong thuật toán tìm kiếm A*, hàm đánh giá tại mỗi trạng thái n được xác định bởi công thức: $f(n) = g(n) + h(n)$

Trong đó g(n) là chi phí thực tế từ trạng thái bắt đầu đến trạng thái hiện tại n (số bước đã di chuyển), và h(n) là chi phí ước lượng từ trạng thái n đến trạng thái đích. Dự án cài đặt hai hàm heuristic kinh điển trong file src/core/heuristics.py để so sánh định lượng.

2.5.1.1. Hàm Heuristic 1: Misplaced Tiles (h1 - Đếm số ô đặt sai vị trí)

- Nguyên lý: Đếm tổng số ô số đang nằm không đúng vị trí của nó so với trạng thái đích (1, 2, ..., N^2 - 1, 0).
- Quy tắc bắt buộc: Không đếm ô trống (số 0). Nếu đếm cả ô trống, khi chỉ còn 1 bước cuối cùng để về đích (1 ô số đổi chỗ với ô trống), giá trị heuristic sẽ bằng 2 thay vì 1, làm mất tính chấp nhận được (admissible).
- Cách cài đặt: Với bàn cờ kích thước N × N biểu diễn bằng tuple 1 chiều state, tại trạng thái đích thì ô ở chỉ số idx ($0 \leq idx < N^2 - 1$) phải mang giá trị idx + 1. Do đó, hàm misplaced_tiles(state, size) duyệt qua tuple và tăng biến đếm thêm 1 mỗi khi gặp phần tử tile != 0 và tile != idx + 1. Độ phức tạp thời gian là $O(N^2)$.

2.5.1.2. Hàm Heuristic 2: Manhattan Distance (h2 – Tổng khoảng cách theo hàng và cột)

- Nguyên lý: Với mỗi ô số tile ≠ 0, tính tổng số bước dịch chuyển tối thiểu theo chiều dọc (hàng) và chiều ngang (cột) để đưa ô đó từ tọa độ hiện tại (r, c) về đúng tọa độ đích (r*, c*) của nó, giả sử không bị các ô khác cản đường.
- Công thức chuyển đổi tọa độ tối ưu trong code: Từ chỉ số mảng 1 chiều idx, tọa độ hiện tại là (r, c) = divmod(idx, size). Vì trạng thái đích xếp tăng dần từ 1 đến N^2 - 1 nên vị trí đích của ô mang giá trị tile (tile ≠ 0) luôn nằm ở chỉ số (tile - 1), suy ra tọa độ đích là (r*, c*) = divmod(tile - 1, size). Công thức này giúp xác định tọa độ đích của mỗi ô trong thời gian hằng số O(1) thay vì phải gọi hàm tìm kiếm goal.index(tile) tốn $O(N^2)$: $h2(n) = \Sigma (|r - r^*| + |c - c^*|)$ *(với mọi ô tile ≠ 0).*

2.5.1.3. Ví dụ minh họa h1(n) và h2(n) trên bàn cờ 8 – Puzzle

Xét trạng thái 8-puzzle (size = 3): state = (8, 1, 3, 4, 0, 2, 7, 6, 5) và trạng thái đích goal = (1, 2, 3, 4, 5, 6, 7, 8, 0):

- Giá trị Misplaced Tiles (h1): Các ô đang nằm đúng vị trí là 3, 4, 7. Các ô nằm sai vị trí (bỏ qua số 0) gồm 5 ô là {8, 1, 2, 6, 5} => h1 = 5.
- Giá trị Manhattan Distance (h2): Khoảng cách từng ô tới vị trí đích: ô 8 (3 bước), ô 1 (1 bước), ô 3 (0 bước), ô 4 (0 bước), ô 2 (2 bước), ô 7 (0 bước), ô 6 (2 bước), ô 5 (2 bước) => h2 = 3 + 1 + 0 + 0 + 2 + 0 + 2 + 2 = 10.

2.5.1.4. Chứng minh tính Chấp nhận được (Admissible) và Quan hệ Trội (Dominance)

Một hàm heuristic h(n) được gọi là chấp nhận được (admissible) nếu nó không bao giờ đánh giá cao hơn chi phí thực tế tối thiểu h*(n) để đi từ n tới đích: $0 \leq h(n) \leq h^*(n)$, *với mọi trạng thái n*

Chứng minh Misplaced Tiles là Admissible: Mỗi bước di chuyển hợp lệ chỉ có thể dịch chuyển đúng 1 ô số vào vị trí ô trống kế cận. Vì vậy, nếu có k ô đang nằm sai vị trí thì cần ít nhất k bước di chuyển mới có thể đưa cả k ô đó về đích. Suy ra $h1(n) \leq h^*(n)$.

Chứng minh Manhattan Distance là Admissible: Mỗi bước di chuyển chỉ làm thay đổi tọa độ hàng hoặc tọa độ cột của đúng 1 ô số thêm tối đa ±1 đơn vị. Do đó, để mỗi ô đi từ (r, c) về (r*, c*), nó bắt buộc phải tốn ít nhất $|r - r^*| + |c - c^*|$ bước. Cộng tổng trên tất cả các ô số (không tính ô trống 0), số bước thực tế h*(n) luôn lớn hơn hoặc bằng tổng khoảng cách Manhattan: $h2(n) \leq h^*(n)$.

Quan hệ trội (Dominance): Mọi ô nằm sai vị trí đều có khoảng cách Manhattan tối thiểu bằng 1 ($|r - r^*| + |c - c^*| \geq 1$), trong khi Misplaced Tiles chỉ cộng đúng 1 cho ô đó. Do đó ta luôn có bất đẳng thức: $h1(n) \leq h2(n) \leq h^*(n)$. Vì Manhattan Distance cung cấp cận dưới sát với chi phí thực tế hơn, thuật toán A* sử dụng Manhattan sẽ cắt tỉa được nhiều nhánh thừa hơn và mở rộng ít node hơn hẳn so với Misplaced Tiles.

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

3.1. Tổng quan dự án PuzzleSolver – AI

PuzzleSolver-AI là chương trình giải bài toán 8-Puzzle và 15-Puzzle sử dụng thuật toán A* kết hợp với các hàm Heuristic.

Hệ thống được xây dựng theo hướng phân chia chức năng thành các module độc lập nhưng có khả năng phối hợp với nhau.

Các thành phần chính gồm:

- Thành phần xử lý trạng thái và thuật toán A*.
- Thành phần Heuristic.
- Thành phần sinh dữ liệu kiểm thử.

- Thành phần đánh giá hiệu năng.
- Thành phần giao diện và hoạt ảnh.

Mục tiêu của kiến trúc này là giúp quá trình xử lý, kiểm thử và hiển thị được tổ chức rõ ràng, thuận tiện cho việc tích hợp và đánh giá.

3.2. Phân tích yêu cầu hệ thống

3.2.1. Yêu cầu chức năng

STT	**Chức năng**	**Mô tả**
1	Biểu diễn trạng thái	Lưu trạng thái dưới dạng Tuple
2	Sinh trạng thái kề	Di chuyển ô trống theo bốn hướng
3	Kiểm tra khả giải	Xác định trạng thái có thể giải hay không
4	Tìm kiếm A*	Tìm đường đi đến trạng thái đích
5	Misplaced Tiles	Tính số ô số sai vị trí
6	Manhattan Distance	Tính tổng khoảng cách Manhattan
7	Random Walk	Sinh trạng thái kiểm thử
8	Benchmark	Đo số node và thời gian thực thi
9	Hiển thị bàn cờ	Vẽ các ô số bằng Pygame
10	Hoạt ảnh	Mô phỏng quá trình trượt ô
11	Điều khiển	Hỗ trợ thao tác bằng bàn phím
12	So sánh	Hiển thị kết quả hai hàm Heuristics

Bảng 1: Yêu cầu chức năng

3.2.2. Yêu cầu phi chức năng

- Tổ chức chương trình thành các module có chức năng rõ ràng.
- Hỗ trợ cả hai kích thước bàn cờ 3 × 3 và 4 × 4.
- Hạn chế việc duyệt lặp lại các trạng thái.
- Đảm bảo khả năng tái lập dữ liệu kiểm thử.
- Có cơ chế giới hạn thời gian và số node mở rộng.
- Giao diện trực quan, dễ quan sát và điều khiển.
- Hỗ trợ hiển thị hoạt ảnh ở tốc độ 60 FPS.

3.3. Phân tích các thành phần chức năng

3.3.1. Thành phần xử lý cốt lõi

Thành phần này đảm nhiệm việc: Biểu diễn trạng thái, xác định vị trí các ô trống, sinh trạng thái kề, kiểm tra điều kiện khả giải, thực hiện thuật toán A*, truy vết và khôi phục đường đi.

3.3.2. Thành phần Heusistic

Bao gồm hai phương pháp:
- Misplaced
- Manhattan Distance

Hai hàm được sử dụng để tính toán giá trị ước lượng h(n), hỗ trợ thuật toán A* lựa chọn trạng thái cần mở rộng.

3.3.3. Thành phần sinh dữ liệu

Sử dụng Random Walk để tạo các trạng thái kiểm thử có thể giải được. Bộ sinh hỗ trợ nhiều mức độ xáo trộn và sử dụng seed cố định nhằm đảm bảo khả năng tái lập.

3.3.4. Thành phần đánh giá hiệu năng

Thực hiện chạy thuật toán trên các bộ dữ liệu kiểm thửu, thu thập: số node mở rộng, độ dài lời giải, thời gian thực thi. Kết quả được lưu vào tệp CSV phục vụ việc phân tích.

3.3.5. Thành phần giao diện

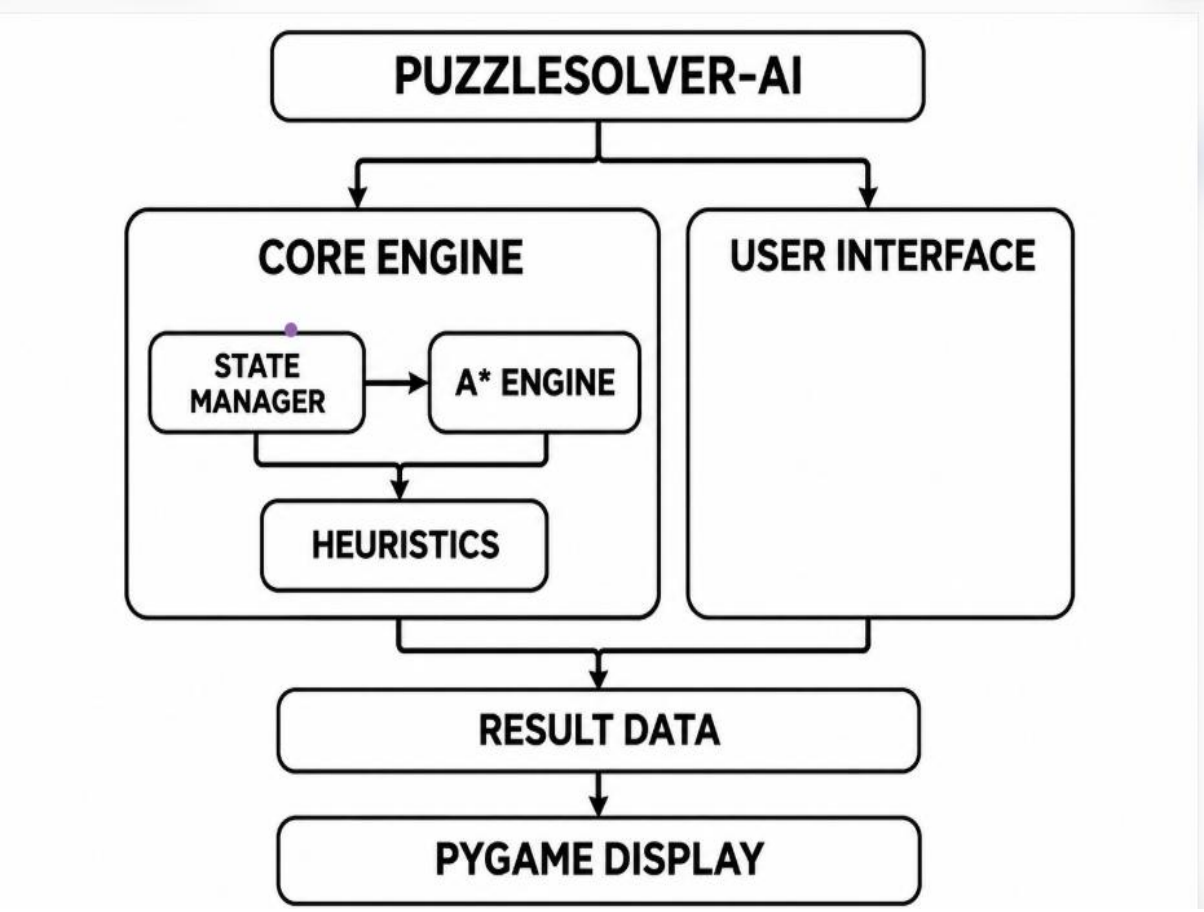
Sử dụng Pygame để hiển thị bàn cờ và quá trình giải. Người dùng có thể tạo bàn cờ mới, thay đổi loại bài toán, thay đổi Heuristic và theo dõi từng bước di chuyển

3.4. Thiết kế kiến trúc hệ thống

Hệ thống được tổ chức theo ba nhóm công việc chính:

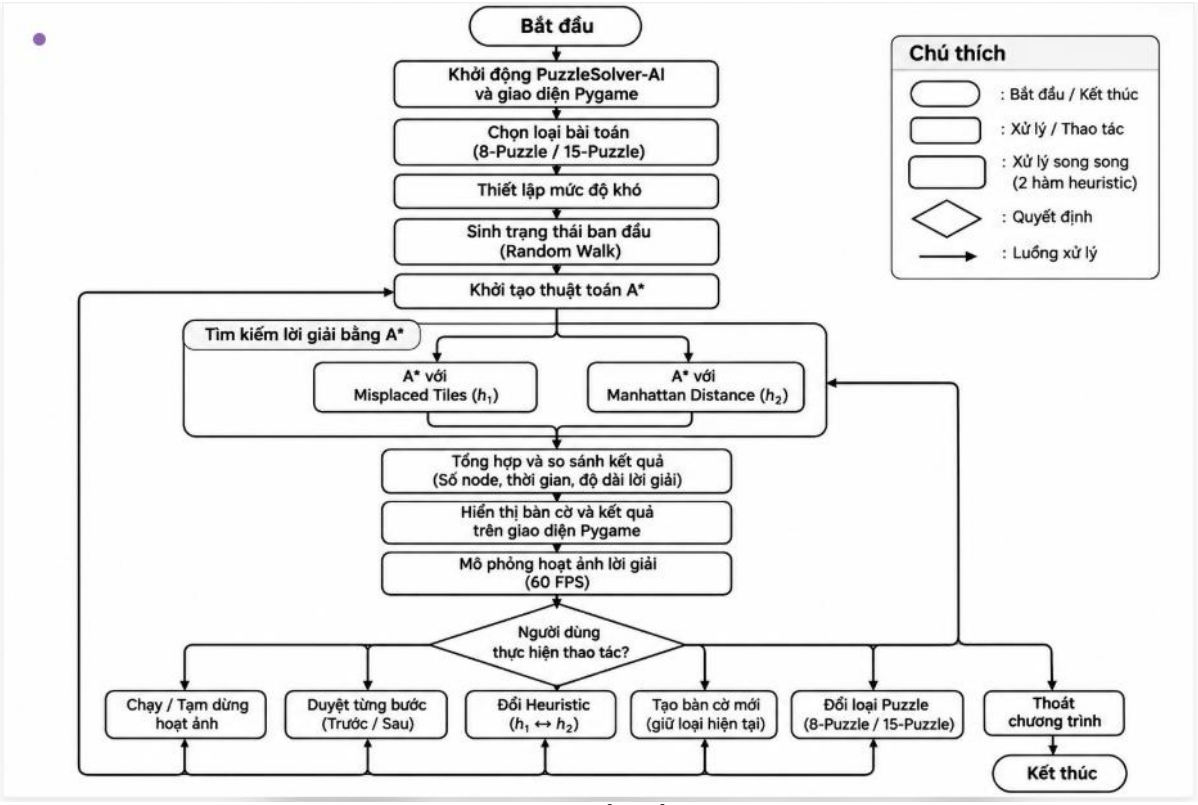
- Nhóm xử lý cốt lõi: phụ trách cơ sở dữ liệu trạng thái, điều kiện khả giải, sinh trạng thái kề và thuật toán A*.
- Nhóm Heuristics và thực nghiệm: phụ trách các hàm Heuristics, sinh dữ liệu Random Walk và đánh giá hiệu năng.
- Nhóm giao diện: phụ trách vẽ bàn cờ, hoạt ảnh Pygame và tích hợp các chức năng thành chương trình hoàn chỉnh.

3.4.1. Sơ đồ kiến trúc tổng thể



Hình 1: Sơ đồ thể hiện mối liên hệ giữa phần xử lý trạng thái, tìm kiếm, Heuristic và giao diện trực quan

3.4.2. Sơ đồ luồng xử lý



Hình 2: Sơ đồ luồng xử lý

3.5. Thiết kế module chương trình

Các module được mô tả trong tài liệu kỹ thuật gồm:

STT	Module	Chức năng
1	src/core/heuristics.py	Cài đặt hai hàm Heristic
2	src/core/generator.py	Sinh dữ liệu Random Walk
3	src/benchmark/run_benchmark.py	Chạy benchmark
4	src/benchmark/compare_heuristics.py	So sánh Heuristic
5	tests/test_heuristics.py	Kiểm thử Heuristic
6	src/demo/board_renderer.py	Vẽ bàn cờ
7	src/demo/main.py	Điều phối giao diện

Bảng 2: Các chức năng xử lý trạng thái và A được cung cấp bởi phần xử lý cốt lõi của dự án*

3.6. Thiết kế dữ liệu đầu vào

Dữ liệu đầu vào của hệ thống bao gồm: kích thước bàn cờ, trạng thái ban đầu, trạng thái đích, số bước Random Walk, phương pháp Heuristic, các giới hạn tài nguyên khi tìm kiếm. Trạng thái được biểu diễn bằng Tupple một chiều.

3.7. Thiết kế dữ liệu đầu ra

Khi tìm kiếm thành công, hệ thống cần thu được: danh sách các trạng thái trên đường đi , số node được mở rộng, thời gian thực thi, độ dài đường đi. Khi tìm kiếm dừng do giới hạn tàu nguyên, chương trình trả về nguyên nhân tương ứng. Dữ liệu thực nghệm được tổng hợp trong tệp benchmark-results.csv.

3.8. Tổng kết chương

Chương 3 đã phân tích các yêu cầu chức năng và thiết kế kiến trúc tổng thể của PuzzleSolver-AI. Việc chia hệ thống thành các thành phần xử lý cốt lõi, Heuristic, sinh dữ liệu, benchmark và giao diện giúp chương trình có cấu trúc rõ ràng, hỗ trợ quá trình phát triển và đánh giá.

CHƯƠNG 4: XÂY DỰNG VÀ TRIỂN KHAI CHƯƠNG TRÌNH

4.1. Môi trường và công cụ phát triển

Dự án sử dụng Python để triển khai các chức năng xử lý và thuật toán. Các thư viện và công cụ được xác định trong tài liệu gồm:

Công cụ / Thư viện	**Vai trò**
Python	Ngôn ngữ lập trình
heapq	Xây dựng hàng đợi ưu tiên Min – Heap
time	Đo thời gian thực thi
Pygame	Xây dựng giao diện và hoạt ảnh
CSV	Lưu dữ liệu kiểm thử và benchmark

Bảng 3: Các công cụ và thư viện

4.2. Xây dựng thuật toán A*

Thuật toán A* là thành phần chính trong PuzzleSolver-AI.

Quá trình cài đặt sử dụng Open Set, Closed Set, g_score và came_from. Khi bắt đầu, trạng thái đầu tiên được đưa vào Open Set với chi phí ban đầu bằng 0. Trong mỗi vòng lặp, chương trình lấy trạng thái có f(n) nhỏ nhất ra khỏi hàng đợi để xử lý. Sau khi kiểm tra trạng thái đích, nếu chưa đạt mục tiêu, chương trình tiến hành sinh các trạng thái kề. Đối với từng trạng thái kề, thuật toán tính chi phí mới và cập nhật nếu tìm được đường đi tốt hơn.

Khi đạt trạng thái đích, hàm reconstruct_path thực hiện truy vết từ trạng thái đích về trạng thái ban đầu để tạo danh sách các bước giải.

4.3. Xây dựng hàm Misplaced Tiles

Misplaced Tiles là hàm Heuristic đếm số lượng ô số đang nằm sai vị trí so với trạng thái đích.

Nguyên lý:

Duyệt toàn bộ các phần tử trong Tuple. Nếu phần tử khác 0 và không bằng giá trị mong đợi tại vị trí tương ứng, biến đếm được tăng lên một đơn vị.

Công thức: *h1(n) = Số lượng ô số nằm sai vị trí.*

Điều kiện quan trọng là không tính ô trống 0.

4.4. Xây dựng hàm Misplaced Tiles

Manhattan Distance tính tổng khoảng cách theo hàng và cột của từng ô số từ vị trí hiện tại đến vị trí đích.
Công thức*: $h2(n) = \Sigma (|r - r^*| + |c - c^*|)$.*

Trong đó:

- (r, c): tọa độ hiện tại
- (r*, c*): tọa độ đích

Ô trống 0 không được tính.

Trong chương trình, tọa độ hiện tại được xác định bằng: Row, col = divmod(idx, size), tọa độ đích được xác định bằng: goal_row, goal_col = divmod(tile – 1, size). Cách tính này cho phép xác định vị trí đích của từng ô số trực tiếp từ giá trị tile.

4.5. Chứng minh tính chất của hai hàm Heuristic

4.5.1. Misplaced Tiles là hàm chấp nhận được

Mỗi lần di chuyển hợp lệ chỉ có thể đưa một ô số vào vị trí thông qua việc hoán đổi với ô trống. Nếu có k ô số nằm sai vị trí, cần ít nhất k bước di chuyển để đưa các ô đó về đúng vị trí.

Do đó: $h1(n) \leq h^*(n)$.

Vì vậy, Misplaced Tiles không đánh giá quá cao chi phí thực tế tối thiểu.

4.5.2. Manhattan Distance là hàm chấp nhận được

Mỗi bước di chuyển chỉ thay đổi tọa độ hàng hoặc cột của một ô số tối đa một đơn vị. Để đưa một ô số từ vị trí hiện tại về vị trí đích, số bước di chuyển tối thiểu không thể nhỏ hơn tổng khoảng cách tuyệt đối giữa hai tọa độ.

Do đó: $h2(n) \leq h^*(n)$.

Manhattan Distance cũng là một hàm Heuristic chấp nhận được.

4.5.3. Quan hệ trội

Với mỗi ô số nằm sai vị trí, khoảng cách Manhattan tối thiểu bằng 1. Trong khi đó, Misplaced Tiles chỉ cộng đúng 1 cho mỗi ô sai vị trí.

Vì vậy: $h1(n) \leq h2(n) \leq h(n).*$

Manhattan Distance cung cấp giá trị ước lượng sát với chi phí thực tế hơn, từ đó giúp A* có khả năng cắt tỉa nhiều nhánh tìm kiếm không cần thiết.

4.6. Xây dựng bộ sinh dữ liệu Random Walk

4.6.1. Lý do sử dụng Random Walk

Nếu sử dụng phương pháp xáo trộn ngẫu nhiên hoàn toàn bằng random.shuffle(), có khoảng 50% khả năng tạo ra trạng thái không thể giải được. Ngoài ra, đối với 15-Puzzle,

cấu hình ngẫu nhiên hoàn toàn thường có thể đòi hỏi số bước giải tối ưu lớn, dẫn đến nguy cơ bùng nổ không gian tìm kiếm.

Vì vậy, dự án sử dụng Random Walk để tạo các trạng thái kiểm thử có thể giải được và kiểm soát mức độ xáo trộn.

4.6.2. Nguyên lý hoạt động

Hàm random_walk_state(size, num_moves, seed) thực hiện:

- Khởi tạo từ trạng thái đích goal_state(size).
- Gọi hàm get_neighbors() để lấy những trạng thái kề hợp lệ.
- Loại bỏ trạng thái liền trước prev_state để tránh thao tác tiến rồi lùi ngay lập tức.
- Chọn ngẫu nhiên một trạng thái kề.
- Lặp lại cho đến khi đạt số bước num_moves.

Vì tất cả trạng thái đều được tạo từ trạng thái đích thông qua những bước di chuyển hợp lệ, trạng thái cuối cùng luôn có thể giải được.

4.6.3. Tính tái lập

Hàm generate_testcases() sử dụng seed = 42.

Các mức độ xáo trộn: {5, 10, 15, 20, 25}. Mỗi mức có 5 trường hợp kiểm thử.

Hai tệp dữ liệu được tạo:

- testcases_8puzzle.csv.
- testcases_15puzzle.csv.

Việc sử dụng seed cố định giúp kết quả thực nghiệm có thể được tái lập trên cùng bộ dữ liệu.

4.7. Xây dựng chức năng Benchmark

Benchmark được sử dụng để đo lường hiệu năng của hai hàm Heuristic.

Chương trình sử dụng ba tiêu chí.

- Số node đã mở rộng (nodes_expanded): Là số trạng thái được lấy khỏi hàng đợi ưu tiên và đưa vào tập đóng để mở rộng.
- Đây là tiêu chí phản ánh khả năng giảm không gian tìm kiếm.
- Độ dài đường đi (solution_length): Được tính bằng: solution_length = len(path) - 1.
- Chỉ số này thể hiện số bước di chuyển cần thiết để đạt trạng thái đích.
- Thời gian thực thi (time_taken): Được đo bằng: time.perf_counter()

Đơn vị tính là giây. Kết quả được tổng hợp vào benchmark_results.csv.

4.8. Xây dựng giao diện trực quan bằng Pygame

4.8.1. Hiển thị trạng thái bàn cờ

Hàm draw_board(screen, state, size, ...) nhận trạng thái Tuple một chiều và chuyển đổi thành lưới hai chiều để hiển thị.

Chương trình sử dụng: *row, col = divmod(idx, size)* Sau đó, hàm _compute_tile_rect() xác định vị trí pixel và kích thước của từng ô trên khu vực board_rect.

4.8.2. Thiết kế màu sắc

Giao diện sử dụng bảng màu Deep Navy (#1C1C28). Các trạng thái ô số được phân biệt như sau:

Loại ô	**Màu sắc**	**Ý nghĩa**
Ô nền	#1C1C28	Nền giao diện
Ô sai vị trí	#2A3044	Chưa đạt vị trí đích
Ô đúng vị trí	#485C84	Đã đúng vị trí
Ô đang di chuyển	#6C80A8	Đang thực hiện hoạt ảnh

Bảng 4: Màu sắc của các ô

Ô trống được hiển thị chìm xuống nền tối. Ô đúng vị trí được bổ sung viền bạc và điểm báo góc trên bên phải.
Thiết kế này giúp người dùng dễ dàng phân biệt trạng thái của các ô số.

4.9. Xây dựng hoạt ảnh trượt ô

Thuật toán A* trả về một danh sách các trạng thái rời rạc: *path = [S0, S1, ..., Sk].*

Để mô phỏng quá trình di chuyển liên tục, hàm animate_transition() thực hiện ba bước.

Bước 1. Xác định ô số di chuyển

Giữa hai trạng thái liên tiếp chỉ có một ô số đổi vị trí với ô trống.

Chương trình xác định vị trí số 0 trong hai trạng thái để tìm ô số cần di chuyển và vị trí bắt đầu, kết thúc của nó.

Bước 2. Nội suy tọa độ

Thời gian trượt của một ô là 170ms.

Chương trình sử dụng hàm Ease-Out Cubic: $e(t) = 1 - (1 - t)^3$. Với t thuộc khoảng [0, 1].

Tọa độ hiện tại được tính:

- curr_row = start_row + (end_row - start_row) × e(t).
- curr_col = start_col + (end_col - start_col) × e(t).

Phương pháp này giúp chuyển động bắt đầu nhanh và giảm tốc dần khi ô số tiến đến vị trí đích.

Bước 3. Vẽ tách lớp

Trong quá trình hoạt ảnh, hàm draw_board() sử dụng tham số skip_tile=moving_tile để ẩn ô số đang di chuyển khỏi lưới tĩnh. Ô số đang di chuyển được vẽ riêng ở lớp phía trên tại tọa độ thực tế. Điều này giúp tránh tình trạng vẽ trùng và tạo ra chuyển động trực quan.

4.10. Tích hợp các thành phần hệ thống

Lớp PuzzleDemoApp trong src/demo/main.py đảm nhiệm việc điều phối hoạt động của chương trình.

- Khi khởi động hoặc khi người dùng nhấn R:
- Gọi random_walk_state() để sinh bàn cờ mới.
- Gọi solve() của thuật toán A* cho hai hàm Heuristic.

- Thu thập chuỗi trạng thái lời giải.
- Hiển thị kết quả so sánh.
- Cho phép người dùng theo dõi các bước di chuyển.

Vòng lặp chính của Pygame duy trì tốc độ 60 FPS, đồng thời tiếp nhận các sự kiện chuột và bàn phím. Chương trình hỗ trợ tự động tính toán lại bố cục thông qua _recalculate_layout() và bật Windows High-DPI Awareness nhằm đảm bảo hình ảnh sắc nét khi trình chiếu trên màn hình có độ phân giải cao.

4.11. Các thao tác điều khiển

Phím	**Chức năng**
R	Tạo bàn cờ mới
H	Đổi Heuristic
Space	Chạy hoặc tạm dừng
M	Chuyển đổi 8 – Puzzle / 15 – Puzzle
Up / Down	Tăng hoặc giảm độ khó
Left / Right	Duyệt từng bước lời giải
F11	Toàn màn hình

Bảng 5: Các thao tác để điều khiển

CHƯƠNG 5: KIỂM THỬ, THỰC NGHIỆM VÀ ĐÁNH GIÁ

5.1. Mục tiêu thực nghiệm

Thực nghiệm được tiến hành nhằm đánh giá hiệu quả của thuật toán A* khi sử dụng hai hàm Heuristic khác nhau.

Các mục tiêu gồm:

- Kiểm tra khả năng tìm kiếm lời giải
- So sánh số lượng node được mở rộng
- So sánh thời gian thực thi
- Đánh giá độ dài lời giải
- Đánh giá khả năng cải thiện hiệu năng của Manhattan Distance
- Quan sát quá trình giải thông qua giao diện Pygame

5.2. Phương pháp kiểm thử

Dự án sử dụng phương pháp kiểm thử trên các trạng thái được sinh bằng Random Walk.

Mỗi bộ dữ liệu gồm năm mức độ xáo trộn: 5 bước, 10 bước, bước, 20 bước, 25 bước. Mỗi mức độ có năm trường hợp kiểm thử.

Tổng cộng có 25 trường hợp cho mỗi loại bài toán và 50 trường hợp cho cả hai loại bài toán.

Mỗi trường hợp được sử dụng để so sánh hai phương pháp:

- Misplaced Tiles
- Manhattan Distance

Seed = 42 được sử dụng để đảm bảo khả năng tái lập dữ liệu.

5.3. Các tiêu chí đánh giá

5.3.1. Số node mở rộng

Nodes_expanded là số trạng thái được lấy ra khỏi hàng đợi ưu tiên và đưa vào tập đóng. Đây là tiêu chí quan trọng nhất trong việc đánh giá khả năng cắt tỉa không gian tìm kiếm của hàm Heuristic.

5.3.2. Độ dài lời giải

Solution_length được tính bằng số bước di chuyển từ trạng thái đầu đến trạng thái đích. Công thức: *solution_length = len(path) - 1.*

5.3.3. Thời gian thực thi

Time_taken được đo bằng time.perf_counter(), tính theo giây. Chỉ tiêu này thể hiện thời gian xử lý thực tế của chương trình.

5.4. Kết quả thực nghiệm đối với 8 – Puzzle

Độ khó	Node Manhattan	Node Misplaced	Thời gian Manhattan	Thời gian Misplaced	Giảm node
5 bước	6,2	7,0	0,00006 s	0,00004 s	11,4 %
10 bước	20,6	49,0	0,00018 s	0,00030 s	58,0 %
15 bước	119,6	410,2	0,00229 s	0,00377 s	70,8 %
20 bước	363,2	2.478,8	0,00265 s	0,01124 s	85,3 %
25 bước	1.463,6	12.677,0	0,00919 s	0,09112 s	88,5 %

Bảng 6: Kết quả trung bình của 8-Puzzle.

5.4.1. Phân tích số node

Ở mức 5 bước, số node được mở rộng giữa hai phương pháp không chênh lệch nhiều.

Manhattan Distance mở rộng trung bình 6,2 node, trong khi Misplaced Tiles mở rộng 7 node.

Khi độ khó tăng lên, sự khác biệt ngày càng rõ rệt.

Tại mức 25 bước:

- Manhattan Distance: 1.463,6 node.
- Misplaced Tiles: 12.677 node.

Tỷ lệ giảm số node đạt 88,5%. Điều này chứng minh Manhattan Distance có khả năng định hướng tìm kiếm tốt hơn trong những trường hợp có độ khó cao.

5.4.2. Phân tích thời gian thực thi

Ở mức 5 bước, Misplaced Tiles có thời gian thực thi thấp hơn. Tuy nhiên, từ mức 10 bước trở lên, Manhattan Distance có thời gian thực thi thấp hơn trong các trường hợp được kiểm thử.

Tại mức 25 bước:

- Manhattan Distance: 0,00919 giây.
- Misplaced Tiles: 0,09112 giây.

Kết quả cho thấy lợi thế về thời gian xử lý của Manhattan Distance khi không gian tìm kiếm tăng lên.

5.5. Kết quả thực nghiệm đối với 15 – Puzzle

Độ khó	Node Manhattan	Node Misplaced	Thời gian Manhattan	Thời gian Misplaced	Giảm node
5 bước	6,0	7,2	0,00012 s	0,00007 s	16,7%
10 bước	15,6	27,6	0,00021 s	0,00024 s	43,5%
15 bước	61,2	199,2	0,00064 s	0,00144 s	69,3%
20 bước	234,4	2.752,4	0,00314 s	0,02391 s	91,5%
25 bước	505,8	10.137,2	0,00707 s	0,09383 s	95,0%

Bảng 7: Kết quả trung bình của 15-Puzzle.

5.5.1. Phân tích số node

Đối với 15-Puzzle, Manhattan Distance cho thấy ưu thế rõ rệt ở những mức độ khó cao. Tại mức 20 bước, Manhattan Distance mở rộng trung bình 234,4 node, trong khi Misplaced Tiles mở rộng 2.752,4 node.

Tỷ lệ giảm đạt 91,5%.

Tại mức 25 bước:

- Manhattan Distance: 505,8 node.
- Misplaced Tiles: 10.137,2 node.

Tỷ lệ giảm số node đạt 95%.

Kết quả này cho thấy Manhattan Distance có khả năng giảm đáng kể số trạng thái cần mở rộng trong bài toán có không gian trạng thái lớn.

5.5.2. Phân tích thời gian thực thi

Ở mức 5 bước, Misplaced Tiles có thời gian thực thi thấp hơn Manhattan Distance. Từ mức 10 bước trở lên, Manhattan Distance có thời gian thực thi thấp hơn trong bộ kiểm thử.

Tại mức 25 bước:

- Manhattan Distance: 0,00707 giây.
- Misplaced Tiles: 0,09383 giây.

Điều này cho thấy việc giảm số node được mở rộng có tác động tích cực đến thời gian thực thi trong các trường hợp kiểm thử có độ khó cao.

5.6. So sánh tổng hợp hai hàm Heuristic

5.6.1. So sánh số node mở rộng

Bài toán	**Độ khó**	**Manhattan**	**Misplaced**	**Tỷ lệ giảm**
8 – Puzzle	5	6,2	7,0	11,4%
8 – Puzzle	10	20,6	49,0	58,0%
8 – Puzzle	15	119,6	410,2	70,8%
8 – Puzzle	20	363,2	2.478,8	85,3%
8 – Puzzle	25	1.463,6	12.677,0	88,5%
15 – Puzzle	5	6,0	7,2	16,7%
15 – Puzzle	10	15,6	27,6	43,5%
15 – Puzzle	15	61,2	199,2	69,3%
15 – Puzzle	20	234,4	2.752,4	91,5%
15 – Puzzle	25	505,8	10.137,2	95,0%

Bảng 8: So sánh số node mở rộng

Qua bảng số liệu, có thể nhận thấy Manhattan Distance luôn mở rộng ít node hơn Misplaced Tiles ở tất cả các mức độ kiểm thử. Tỷ lệ giảm số node tăng lên khi mức độ xáo trộn tăng.

5.6.2. So sánh thời gian thực thi

Bài toán	**Độ khó**	**Manhattan**	**Misplaced**
8 – Puzzle	5	0,00006 s	0,00004 s
8 – Puzzle	10	0,00018 s	0,00030 s
8 – Puzzle	15	0,00229 s	0,00377 s

8 – Puzzle	20	0,00265 s	0,01124 s
8 – Puzzle	25	0,00919 s	0,09112 s
15 – Puzzle	5	0,00012 s	0,00007 s
15 – Puzzle	10	0,00021 s	0,00024 s
15 – Puzzle	15	0,00064 s	0,00144 s
15 – Puzzle	20	0,00314 s	0,02391 s
15 – Puzzle	25	0,00707 s	0,09383 s

Bảng 9: So sánh thời gian thực thi

Manhattan Distance có thời gian thực thi thấp hơn trong phần lớn trường hợp kiểm thử, đặc biệt ở những mức độ khó cao.

Tuy nhiên, tại mức 5 bước, Misplaced Tiles có thời gian thực thi thấp hơn ở cả hai dạng bài toán.

Điều này cho thấy với bài toán đơn giản, chi phí tính toán của hàm Heuristic cũng là yếu tố cần xem xét.

5.7. Đánh giá độ dài thời gian

Theo kết quả thực nghiệm được ghi nhận trong tài liệu kỹ thuật, cả hai hàm Heuristic đều tìm ra lời giải có độ dài ngắn nhất bằng nhau trên các trường hợp giải thành công.

Điều này phù hợp với tính chất chấp nhận được của hai hàm Heuristic khi được sử dụng trong thuật toán A*.

Như vậy, sự khác biệt chủ yếu giữa hai phương pháp không nằm ở chất lượng tối ưu của đường đi mà nằm ở hiệu quả tìm kiếm và số lượng node cần mở rộng.

5.8. Đánh giá giao diện trực quan

Giao diện Pygame giúp người dùng quan sát trực tiếp trạng thái bàn cờ và quá trình di chuyển các ô số. Các ô số được phân biệt bằng màu sắc theo trạng thái, giúp nhận biết ô đúng vị trí, ô sai vị trí, ô trống và ô đang di chuyển.

Hoạt ảnh sử dụng Ease-Out Cubic trong khoảng thời gian 170ms, giúp quá trình trượt ô diễn ra tự nhiên và mượt mà. Vòng lặp chính hoạt động ở tốc độ 60 FPS, hỗ trợ các thao tác điều khiển bằng bàn phím.

Giao diện cũng cho phép thay đổi loại bài toán, thay đổi Heuristic, tạo bàn cờ mới, chạy hoặc tạm dừng và duyệt từng bước lời giải.

Nhờ đó, người dùng có thể quan sát trực quan cách thuật toán tìm kiếm và sự khác biệt giữa hai phương pháp Heuristic.

5.9. Nhận xét chung

Từ kết quả thực nghiệm, có thể đưa ra những nhận xét sau:

- Thuật toán A* có khả năng tìm kiếm lời giải tối ưu đối với các trường hợp giải thành công trong bộ kiểm thử.
- Cả hai hàm Misplaced Tiles và Manhattan Distance đều thỏa mãn tính chấp nhận được.
- Manhattan Distance luôn mở rộng ít node hơn Misplaced Tiles trong bộ dữ liệu thực nghiệm.
- Hiệu quả của Manhattan Distance thể hiện rõ hơn khi mức độ khó tăng lên.
- Cơ chế giới hạn node và thời gian giúp chương trình kiểm soát quá trình tìm kiếm trong những trường hợp không gian trạng thái lớn.
- Giao diện Pygame hỗ trợ tốt việc trình diễn và quan sát quá trình giải bài toán.

CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết quả đạt được

Sau quá trình nghiên cứu, thiết kế và triển khai, dự án PuzzleSolver-AI đã đạt được những kết quả chính sau:

Về lý thuyết:

- Nghiên cứu mô hình không gian trạng thái của 8-Puzzle và 15-Puzzle.
- Tìm hiểu điều kiện khả giải dựa trên số nghịch thế.
- Phân tích nguyên lý hoạt động của thuật toán A*.
- Tìm hiểu tính chấp nhận được của hàm Heuristic.

Về thuật toán:

- Triển khai tìm kiếm A*.
- Xây dựng hàm Misplaced Tiles.

- Xây dựng hàm Manhattan Distance.
- Xây dựng cơ chế truy vết đường đi.
- Bổ sung giới hạn node và thời gian thực thi.

Về dữ liệu và thực nghiệm:

- Xây dựng bộ sinh dữ liệu Random Walk.
- Sử dụng seed cố định để tái lập kết quả.
- Thực hiện benchmark trên hai dạng bài toán.
- So sánh hiệu năng hai hàm Heuristic.

Về giao diện:

- Xây dựng giao diện Pygame.
- Hiển thị trạng thái bàn cờ.
- Phân biệt các ô theo trạng thái.
- Mô phỏng hoạt ảnh trượt ô.
- Hỗ trợ điều khiển quá trình giải.

6.2. Ưu điểm của hệ thống

Hệ thống PuzzleSolver-AI có những ưu điểm:

- Hỗ trợ hai dạng bài toán 8-Puzzle và 15-Puzzle.
- Sử dụng thuật toán A* để tìm lời giải tối ưu.
- Triển khai hai hàm Heuristic có thể so sánh trực tiếp.
- Có khả năng sinh dữ liệu kiểm thử tái lập.
- Có cơ chế giới hạn tài nguyên.
- Có giao diện trực quan và hoạt ảnh.
- Có thể thu thập số liệu để đánh giá hiệu năng.

6.3. Hạn chế của hệ thống

Mặc dù đạt được các mục tiêu chính, hệ thống vẫn có một số hạn chế.

Thứ nhất, không gian trạng thái của 15-Puzzle rất lớn. Khi cấu hình có độ khó cao, thuật toán A* có thể phải mở rộng nhiều trạng thái và tiêu tốn đáng kể bộ nhớ.

Thứ hai, thuật toán A* sử dụng các cấu trúc dữ liệu để lưu trữ trạng thái đã duyệt và trạng thái chờ mở rộng. Điều này làm tăng nhu cầu sử dụng RAM khi không gian tìm kiếm lớn.

Thứ ba, bộ dữ liệu kiểm thử hiện tại tập trung vào năm mức Random Walk từ 5 đến 25 bước, chưa đại diện cho toàn bộ các cấu hình có thể xuất hiện.

Thứ tư, thời gian thực thi phụ thuộc vào môi trường phần cứng và phần mềm. Do đó, cần ghi nhận cấu hình máy khi tiến hành các thực nghiệm mở rộng.

6.4. Hướng phát triển

Nâng cao thuật toán tìm kiếm: Nghiên cứu và thử nghiệm những phương pháp tìm kiếm khác nhằm giảm chi phí bộ nhớ và thời gian xử lý đối với 15-Puzzle.

Cải thiện hàm Heuristic: Nghiên cứu các hàm Heuristic nâng cao để tăng khả năng định hướng tìm kiếm và giảm số lượng node phải mở rộng.

Mở rộng bộ dữ liệu thực nghiệm: Tạo thêm các bộ dữ liệu với mức độ khó cao hơn, tăng số lượng trường hợp kiểm thử và thực hiện đánh giá trên nhiều cấu hình khác nhau.

Phát triển giao diện: Cải thiện giao diện Pygame, bổ sung thêm các chức năng thống kê và trình diễn quá trình tìm kiếm.

Mở rộng khả năng so sánh: Bổ sung thêm các thuật toán tìm kiếm khác để đánh giá sự khác biệt về hiệu năng, chất lượng lời giải và khả năng sử dụng tài nguyên.

6.5. Kết luận chung

Đề tài PuzzleSolver-AI – Bộ giải bài toán 8-Puzzle / 15-Puzzle bằng thuật toán A* đã tập trung nghiên cứu và triển khai một hệ thống giải bài toán N-Puzzle thông qua phương pháp tìm kiếm có thông tin.

Trong quá trình thực hiện, nhóm đã xây dựng mô hình biểu diễn trạng thái, kiểm tra điều kiện khả giải, triển khai thuật toán A* và hai hàm Heuristic Misplaced Tiles, Manhattan Distance.

Bên cạnh đó, phương pháp Random Walk được sử dụng để sinh các trạng thái kiểm thử có thể giải được và đảm bảo khả năng tái lập. Các tiêu chí benchmark giúp đánh giá định lượng hiệu quả của từng phương pháp Heuristic.

Kết quả thực nghiệm cho thấy Manhattan Distance có khả năng giảm đáng kể số lượng node mở rộng so với Misplaced Tiles. Đặc biệt, tại mức 25 bước Random Walk, tỷ lệ giảm số node đạt 88,5% đối với 8-Puzzle và 95% đối với 15-Puzzle.

Ngoài phần thuật toán, giao diện Pygame giúp trực quan hóa quá trình giải, hỗ trợ quan sát từng bước di chuyển và so sánh hiệu năng giữa hai phương pháp Heuristic.

Thông qua dự án, nhóm đã vận dụng được các kiến thức về Trí tuệ nhân tạo, tìm kiếm trạng thái, cấu trúc dữ liệu, lập trình và kiểm thử phần mềm vào việc xây dựng một sản phẩm cụ thể.

Nhìn chung, PuzzleSolver-AI đã đạt được các mục tiêu chính trong phạm vi nghiên cứu và triển khai, đồng thời tạo nền tảng để tiếp tục phát triển các phương pháp tìm kiếm hiệu quả hơn cho những bài toán có không gian trạng thái lớn.

TÀI LIỆU THAM KHẢO

[1] *Tài liệu kỹ thuật phần việc Thành viên 4, dự án PuzzleSolver-AI: Không gian trạng thái và biểu diễn bài toán N-Puzzle; lý thuyết điều kiện khả giải; thuật toán tìm kiếm A*.*

[2] *Tài liệu kỹ thuật phần việc Thành viên 5 và Thành viên 6, dự án PuzzleSolver-AI: Heuristic, sinh đề, đánh giá hiệu năng, giao diện trực quan và hoạt ảnh Pygame.*

[3] *Mã nguồn dự án PuzzleSolver-AI: Các module xử lý Heuristic, sinh dữ liệu, benchmark và giao diện trực quan.*

PHỤ LỤC

PHỤ LỤC A: DANH SÁCH MODULE MÃ NGUỒN

STT	**Đường dẫn**	**Nội dung**
1	src/core/heuristics.py	Hai hàm Heuristic
2	src/core/generator.py	Sinh trạng thái Random Walk
3	src/benchmark/run_benchmark.py	Chạy benchmark
4	src/benchmark/compare_heuristics.py	So sánh Heuristic
5	tests/test_heuristics.py	Kiểm thử Heuristic
6	src/demo/board_renderer.py	Vẽ bàn cờ
7	src/demo/main.py	giao diện chính